

Язык программирования C++

Лекция 22: динамически загружаемые модули
(плагины), библиотека Boost, работа с сетью

Лисица Никита

2026

- При линковке программы можно линковать *статические* и *динамические библиотеки*

- При линковке программы можно линковать *статические* и *динамические библиотеки*
- Код из *статических* библиотек (в бинарном виде!) копируется в наш исполняемый файл

- При линковке программы можно линковать *статические* и *динамические библиотеки*
- Код из *статических* библиотек (в бинарном виде!) копируется в наш исполняемый файл
- На код из *динамических* библиотек ссылается наш исполняемый файл

- При линковке программы можно линковать *статические* и *динамические библиотеки*
- Код из *статических* библиотек (в бинарном виде!) копируется в наш исполняемый файл
- На код из *динамических* библиотек ссылается наш исполняемый файл
- При запуске исполняемого файла операционная система сама подгружает необходимые динамические библиотеки

- Иногда хочется в процессе работы программы (а не при старте!) загрузить заранее неизвестную библиотеку

- Иногда хочется в процессе работы программы (а не при старте!) загрузить заранее неизвестную библиотеку
- Как правило, это т.н. *плагин* – дополнительная, заранее не предусмотренная функциональность программы

- Иногда хочется в процессе работы программы (а не при старте!) загрузить заранее неизвестную библиотеку
- Как правило, это т.н. *плагин* – дополнительная, заранее не предусмотренная функциональность программы
- Некоторые системные библиотеки иногда тоже загружаются этим способом (напр. загрузчики OpenGL/Vulkan)

- Иногда хочется в процессе работы программы (а не при старте!) загрузить заранее неизвестную библиотеку
- Как правило, это т.н. *плагин* – дополнительная, заранее не предусмотренная функциональность программы
- Некоторые системные библиотеки иногда тоже загружаются этим способом (напр. загрузчики OpenGL/Vulkan)
- Как это сделать?

- Загрузить файл динамической библиотеки по заданному пути: `dlopen` (Linux, MacOS) / `LoadLibraryW` (Windows)

- Загрузить файл динамической библиотеки по заданному пути: `dlopen` (Linux, MacOS) / `LoadLibraryW` (Windows)
- Возвращает некий идентификатор библиотеки

- Загрузить файл динамической библиотеки по заданному пути: `dlopen` (Linux, MacOS) / `LoadLibraryW` (Windows)
- Возвращает некий идентификатор библиотеки
- Когда библиотека больше не нужна, её следует отгрузить: `dlclose` (Linux, MacOS) / `FreeLibrary` (Windows)

- Загрузить из динамической библиотеки функцию по её имени: `dlsym` (Linux, MacOS) / `GetProcAddress` (Windows)

Плагины: загрузка функций

- Загрузить из динамической библиотеки функцию по её имени: `dlsym` (Linux, MacOS) / `GetProcAddress` (Windows)
- **NB:** ожидается имя функции после *name mangling*'а! В C++ коде лучше всего занести экспортируемую функцию в `extern "C"`

Плагины: загрузка функций

- Загрузить из динамической библиотеки функцию по её имени: `dlsym` (Linux, MacOS) / `GetProcAddress` (Windows)
- **NB:** ожидается имя функции после *name mangling*'а! В C++ коде лучше всего занести экспортируемую функцию в `extern "C"`
- Возвращает `void *`, ничего не знает о настоящем типе функции!

Плагины: загрузка функций

- Загрузить из динамической библиотеки функцию по её имени: `dlsym` (Linux, MacOS) / `GetProcAddress` (Windows)
- **NB:** ожидается имя функции после *name mangling*'а! В C++ коде лучше всего занести экспортируемую функцию в `extern "C"`
- Возвращает `void *`, ничего не знает о настоящем типе функции!
- Чтобы вызвать функцию, придётся явно привести тип к указателю на функцию:

```
// Код программы
auto func = static_cast<void(int)>(dlsym(library, "myfunc"));
func(42);
```


- Неудобно помнить о типах большого числа функций

- Неудобно помнить о типах большого числа функций
- Если не угадать тип функции, произойдёт UB (вероятнее всего, креш)

- Неудобно помнить о типах большого числа функций
- Если не угадать тип функции, произойдёт UB (вероятнее всего, креш)
- Приходится заранее договариваться об *интерфейсе* плагина

Плагины: интерфейс через классы

- Удобно выделить интерфейс плагина в виде ООП-интерфейса, т.е. класса с виртуальными функциями:

```
// Общий header программы и плагина
class IPlugin {
public:
    void initialize(const ProgramContext&) = 0;
    void apply(Data&) = 0;
    void serialize(const Data&, std::ostream&) = 0;

    std::string name() const = 0;
    std::string describe() const = 0;

    ~IPlugin() = 0;
};
```

Плагины: интерфейс через классы

- Тогда в самом плагине достаточно сделать функцию, создающую реализацию интерфейса:

```
// Код плагина
class MyPlugin : public IPlugin { ... };

extern "C" IPlugin* create() {
    return new MyPlugin();
}
```

Плагины: интерфейс через классы

- Для загрузки плагина нужно загрузить эту функцию и вызвать:

```
// Код программы
auto create = static_cast<IPlugin*>(<
    dlsym(library, "create"));

IPlugin* plugin = create();
plugin->initialize();

std::cout << "Loaded plugin "
    << plugin->name() << std::endl;
```

- В общем случае, нет гарантии, что динамическая память (куча) процесса и динамической библиотеки – это одно и то же

Плагины: удаление

- В общем случае, нет гарантии, что динамическая память (куча) процесса и динамической библиотеки – это одно и то же
- $\implies$ Объекты, выделенные через `malloc/new` в динамической библиотеке, нужно удалять через `free/delete` в коде библиотеки

```
// Код плагина

extern "C" IPlugin* create() {
    return new MyPlugin();
}

extern "C" void destroy(IPlugin* plugin) {
    delete plugin;
}
```


- Удобно выделять объекты внутри плагина через `std::make_shared`: он запомнит *deleter* при создании объекта, и подцепит функцию удаления из самого плагина

- Удобно выделять объекты внутри плагина через `std::make_shared`: он запомнит *deleter* при создании объекта, и подцепит функцию удаления из самого плагина
- **NB:** Сам интерфейс плагина (`IPlugin`) так не создать: нетривиальные C++-объекты нельзя передавать/возвращать через `extern "C"` функции

Плагины: shared_ptr

- Удобно выделять объекты внутри плагина через `std::make_shared`: он запомнит *deleter* при создании объекта, и подцепит функцию удаления из самого плагина
- **NB:** Сам интерфейс плагина (`IPlugin`) так не создать: нетривиальные C++-объекты нельзя передавать/возвращать через `extern "C"` функции

```
// Код плагина
class MyPlugin : public IPlugin {
public:
    std::shared_ptr<Object> makeObject() override {
        return std::make_shared<Object>();
    }
};
```

- Экспортируем минимум функций

Плагины: интерфейс через классы

- Экспортируем минимум функций
- Все остальные функции плагина работают через понятный компилятору C++ интерфейс $\implies$ сложнее ошибиться в использовании плагина

Плагины: интерфейс через классы

- Экспортируем минимум функций
- Все остальные функции плагина работают через понятный компилятору C++ интерфейс $\implies$ сложнее ошибиться в использовании плагина
- Фактически, мы заменяем импортное неизвестных функций на механизм виртуальных функций C++

- Boost – самый крупный набор библиотек общего назначения для C++

- Boost – самый крупный набор библиотек общего назначения для C++
- Обычно содержит код очень высокого качества, сравнимый со стандартной библиотекой, и работающий под большинством компиляторов

- Boost – самый крупный набор библиотек общего назначения для C++
- Обычно содержит код очень высокого качества, сравнимый со стандартной библиотекой, и работающий под большинством компиляторов
- Многие части стандартной библиотеки пришли из Boost: `any`, `array`, `shared_ptr`, `chrono`, ...

- Boost – самый крупный набор библиотек общего назначения для C++
- Обычно содержит код очень высокого качества, сравнимый со стандартной библиотекой, и работающий под большинством компиляторов
- Многие части стандартной библиотеки пришли из Boost: `any`, `array`, `shared_ptr`, `chrono`, ...
- Часто библиотеки из Boost ругают за излишнюю обобщённость (Boost.Geometry, Boost.GIL)

- Релизится 3 раза в год, первая версия вышла в 1999 году, последняя – 3 апреля 2025 (версия 1.88.0)

- Релизится 3 раза в год, первая версия вышла в 1999 году, последняя – 3 апреля 2025 (версия 1.88.0)
- `boost.org`

- Релизится 3 раза в год, первая версия вышла в 1999 году, последняя – 3 апреля 2025 (версия 1.88.0)
- **boost.org**
- Список библиотек: **`boost.org/libraries/latest/grid`**

- IP (Internet Protocol) – протокол сетевого уровня, связывающий разные компьютеры в сеть с помощью IP-адресов

- IP (Internet Protocol) – протокол сетевого уровня, связывающий разные компьютеры в сеть с помощью IP-адресов
- TCP (Transmission Control Protocol) – протокол транспортного уровня, обеспечивающий общение непрерывными потоками данных, с гарантиями доставки

- IP (Internet Protocol) – протокол сетевого уровня, связывающий разные компьютеры в сеть с помощью IP-адресов
- TCP (Transmission Control Protocol) – протокол транспортного уровня, обеспечивающий общение непрерывными потоками данных, с гарантиями доставки
- UDP (User Datagram Protocol) – протокол транспортного уровня, обеспечивающий общение пакетами данных, без каких-либо гарантий

- TCP гарантирует доставку данных и работает с ними в виде потока, это усложняет его реализацию и требует дополнительной синхронизации, приводящей к задержкам

- TCP гарантирует доставку данных и работает с ними в виде потока, это усложняет его реализацию и требует дополнительной синхронизации, приводящей к задержкам
 - Весь интернет
 - Скачивание файлов
 - Общение между серверами

- TCP гарантирует доставку данных и работает с ними в виде потока, это усложняет его реализацию и требует дополнительной синхронизации, приводящей к задержкам
 - Весь интернет
 - Скачивание файлов
 - Общение между серверами
- UDP ничего не гарантирует $\implies$ он быстрее, но им сложнее пользоваться
 - Мультиплеерные игры
 - Мультимедия (напр. стриминг видео)

- В большинстве ОС есть реализация POSIX сокетов – интерфейса для работы с TCP/UDP

- В большинстве ОС есть реализация POSIX сокетов – интерфейса для работы с TCP/UDP
- Сокеты – особый тип файлового дескриптора, т.е. просто идентификатор типа `int`

- Создать сокет и присоединиться к серверу по IP:

```
// SOCK_STREAM - TCP
// SOCK_DGRAM - UDP
int my_socket = socket(AF_INET, SOCK_STREAM, 0);

sockaddr_in remote_address{};
remote_address.sin_family = AF_INET;
// Порт в network byte order
remote_address.sin_port = htons(80);
inet_pton(AF_INET, "23.215.0.138", &remote_address.sin_addr);

connect(my_socket, (sockaddr*)&remote_address,
        sizeof(remote_address));
```

- Отправка и получение данных – как с файлами:

```
const char * message = ...;  
write(my_socket, message, strlen(message));  
  
char buffer[1024];  
read(my_socket, buffer, sizeof(buffer));
```

- Отправка и получение данных – как с файлами:

```
const char * message = ...;  
write(my_socket, message, strlen(message));  
  
char buffer[1024];  
read(my_socket, buffer, sizeof(buffer));
```

- Это блокирующие операции!

- Привязать сокет к порту:

```
sockaddr_in server_address{};  
server_address.sin_family = AF_INET;  
server_address.sin_addr.s_addr = INADDR_ANY;  
server_address.sin_port = htons(8080);  
  
bind(my_socket, (struct sockaddr*)&server_address,  
    sizeof(server_address));  
listen(my_socket, 3);
```

- Принимать входящие соединения:

```
sockaddr_in client_address{};
socklen_t client_addrlen = sizeof(client_address);

while (true)
{
    int client_fd = accept(socket_fd,
        (struct sockaddr*)&client_address,
        &client_addrlen);

    // Общаемся через client_fd ...

    close(client_fd);
}
```

- Принимать входящие соединения:

```
sockaddr_in client_address{};
socklen_t client_addrlen = sizeof(client_address);

while (true)
{
    int client_fd = accept(socket_fd,
        (struct sockaddr*)&client_address,
        &client_addrlen);

    // Общаемся через client_fd ...

    close(client_fd);
}
```

- **NB:** Такой способ умеет работать только с одним клиентом одновременно

- Часто программе нужно работать с несколькими соединениями параллельно

Сеть: много соединений

- Часто программе нужно работать с несколькими соединениями параллельно
- Банковское приложение: общается сразу с несколькими серверами

- Часто программе нужно работать с несколькими соединениями параллельно
- Банковское приложение: общается сразу с несколькими серверами
- База данных: отвечает сразу на несколько параллельных запросов

- Часто программе нужно работать с несколькими соединениями параллельно
- Банковское приложение: общается сразу с несколькими серверами
- База данных: отвечает сразу на несколько параллельных запросов
- Сервер онлайн-игры: отвечает сразу нескольким игрокам

- Часто программе нужно работать с несколькими соединениями параллельно
- Банковское приложение: общается сразу с несколькими серверами
- База данных: отвечает сразу на несколько параллельных запросов
- Сервер онлайн-игры: отвечает сразу нескольким игрокам
- И т.д.

Сеть: много соединений

- Часто программе нужно работать с несколькими соединениями параллельно
- Банковское приложение: общается сразу с несколькими серверами
- База данных: отвечает сразу на несколько параллельных запросов
- Сервер онлайн-игры: отвечает сразу нескольким игрокам
- И т.д.
- Есть два основных способа это реализовать: потоки и асинхронные соединения

- Создаём по потоку на каждое соединение

- Создаём по потоку на каждое соединение
- Нужна синхронизация между потоками (мьютексы, атомики, и т.д.)

- Создаём по потоку на каждое соединение
- Нужна синхронизация между потоками (мьютексы, атомики, и т.д.)
- Переключение между потоками дорогое – если соединений слишком много, будет много накладных расходов

- Асинхронная работа с сокетами требует поддержки со стороны ОС

- Асинхронная работа с сокетами требует поддержки со стороны ОС
- Linux: `epoll`
- Windows: `iocp` (I/O Completion Ports)
- MacOS, FreeBSD: `kqueue`

- Асинхронная работа с сокетами требует поддержки со стороны ОС
- Linux: `epoll`
- Windows: `iocp` (I/O Completion Ports)
- MacOS, FreeBSD: `kqueue`
- Идея: отдаём ОС большой набор сокетов, и просим нас 'разбудить', когда на какой-нибудь из них придут данные

- Потоки при этом использовать не обязательно (но можно)

- Потоки при этом использовать не обязательно (но можно)
- Минимум накладных расходов, всё ожидание и синхронизацию за нас делает ОС

- Обычно это оборачивают в интерфейс, работающий с *callback*'ами (например, лямбда-функциями)

Сеть: асинхронные соединения

- Обычно это оборачивают в интерфейс, работающий с *callback*'ами (например, лямбда-функциями)
- Пример: отправь данные в сокет, а когда придёт ответ, вызови такую-то функцию:

```
socket.send(my_data, [ε]{  
    socket.receive(buffer, [ε]{  
        // handle data in buffer  
    });  
});
```

Сеть: асинхронные соединения

- Обычно это оборачивают в интерфейс, работающий с *callback*'ами (например, лямбда-функциями)
- Пример: отправь данные в сокет, а когда придёт ответ, вызови такую-то функцию:

```
socket.send(my_data, [ε]{  
    socket.receive(buffer, [ε]{  
        // handle data in buffer  
    });  
});
```

- Такой код – когда результат функции готов не сразу, а когда-нибудь – называют *асинхронным*

- Асинхронность – стандартный сегодня способ работы с сетью

- Асинхронность – стандартный сегодня способ работы с сетью
- Вместо callback'ов удобнее использовать *корутины* – они позволяют писать асинхронный код так же, как императивный

- Асинхронность – стандартный сегодня способ работы с сетью
- Вместо callback'ов удобнее использовать *корутины* – они позволяют писать асинхронный код так же, как императивный
- Node JS, Python asyncio, Boost.Asio, C++20 coroutines, etc

- Asio – библиотека для асинхронной работы с сетью в C++

- Asio – библиотека для асинхронной работы с сетью в C++
- Входит в Boost, но есть и standalone вариант (т.е. не зависящий от Boost'a)

- Asio – библиотека для асинхронной работы с сетью в C++
- Входит в Boost, но есть и standalone вариант (т.е. не зависящий от Boost'a)
- На её основе разрабатывается Networking TS – работа с сетью в стандартной библиотеке C++

- Asio – библиотека для асинхронной работы с сетью в C++
- Входит в Boost, но есть и standalone вариант (т.е. не зависящий от Boost'a)
- На её основе разрабатывается Networking TS – работа с сетью в стандартной библиотеке C++
- Поддерживает корутины из C++20

- Пример создания сокета и отправки данных:

```
using boost::asio;
using boost::asio::ip::tcp;

asio::io_context io_context;

tcp::socket socket(io_context);
connect(socket, address);
write(socket, buffer(message, sizeof(message)));
read(socket, buffer(answer, sizeof(answer)));
```